Day-8 - Take-Home - Assignment

Part A:- Student Admission System

File Link:-
https://github.com/NiranjanHebli/Day8-AM-Assigment/blob/main/student_admission_system.py

Output Screenshots:-

```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-AM-Assigment % python3 student
_admission_system.py

+-----------------------------------------------+
|   UNIVERSITY ADMISSION SCREENING TOOL         |
+-----------------------------------------------+

Enter student details below.

Entrance Score (0-100)      : 96
GPA (0-10)                  : 7.8
Recommendation Letter (yes/no): no
Category (general/obc/sc_st): general
Extracurricular Score (0-10): 0

====================================================
        UNIVERSITY ADMISSION DECISION REPORT
====================================================
  Entrance Score       : 96.0
  GPA                  : 7.8
  Recommendation       : No
  Category             : GENERAL
  Extracurricular Score: 0.0
----------------------------------------------------
  Bonus Applied        : None (scholarship threshold met — bonuses not required)
  Effective Score      : 96.0
----------------------------------------------------
  Result  =>  ADMITTED (Scholarship)
  Reason  : Entrance score 96.0 >= 95. Auto-admitted with scholarship regardless o
f other criteria.
====================================================

Evaluate another student? (yes/no): 
```

```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-AM-Assigment % python3 student
_admission_system.py
  Category             : GENERAL
  Extracurricular Score: 0.0
----------------------------------------------------
  Bonus Applied        : None (scholarship threshold met — bonuses not required)
  Effective Score      : 96.0
----------------------------------------------------
  Result  =>  ADMITTED (Scholarship)
  Reason  : Entrance score 96.0 >= 95. Auto-admitted with scholarship regardless o
f other criteria.
====================================================

Evaluate another student? (yes/no): yes

Entrance Score (0-100)      : 79
GPA (0-10)                  : 8
Recommendation Letter (yes/no): yes
Category (general/obc/sc_st): obc
Extracurricular Score (0-10): 10

====================================================
        UNIVERSITY ADMISSION DECISION REPORT
====================================================
  Entrance Score       : 79.0
  GPA                  : 8.0
  Recommendation       : Yes
  Category             : OBC
  Extracurricular Score: 10.0
----------------------------------------------------
  Bonus Applied        : +5 (recommendation)  +3 (extracurricular)
  Effective Score      : 87.0
----------------------------------------------------
  Result  =>  ADMITTED (Regular)
  Reason  : Meets OBC cutoff (87.0 >= 65) and GPA requirement (8.0 >= 7.0)
====================================================
```

```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-AM-Assigment % python3 student
_admission_system.py
--------------------------------------------------
  Result  =>  ADMITTED (Regular)
  Reason  : Meets OBC cutoff (87.0 >= 65) and GPA requirement (8.0 >= 7.0)
==================================================

Evaluate another student? (yes/no): yes

Entrance Score (0-100)      : -1
  x Value out of range. Must be between 0 and 100.
Entrance Score (0-100)      : 56
GPA (0-10)                  : 100
  x Value out of range. Must be between 0 and 10.
GPA (0-10)                  : 6.3
Recommendation Letter (yes/no): no
Category (general/obc/sc_st): sc_st
Extracurricular Score (0-10): 5

==================================================
        UNIVERSITY ADMISSION DECISION REPORT
==================================================
  Entrance Score       : 56.0
  GPA                  : 6.3
  Recommendation       : No
  Category             : SC/ST
  Extracurricular Score: 5.0
--------------------------------------------------
  Bonus Applied        : None
  Effective Score      : 56.0
--------------------------------------------------
  Result  =>  WAITLISTED
  Reason  : Effective score 56.0 meets the SC/ST cutoff (55), but GPA 6.3 is below
 the minimum of 7.0.
==================================================

Evaluate another student? (yes/no):
```

```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-AM-Assigment % python3 student
_admission_system.py
--------------------------------------------------
  Bonus Applied        : +5 (recommendation)
  Effective Score      : 81.0
--------------------------------------------------
  Result  =>  WAITLISTED
  Reason  : Effective score 81.0 meets the SC/ST cutoff (55), but GPA 5.0 is below
 the minimum of 7.0.
==================================================

Evaluate another student? (yes/no): yes

Entrance Score (0-100)      : 20
GPA (0-10)                  : 8
Recommendation Letter (yes/no): no
Category (general/obc/sc_st): obc
Extracurricular Score (0-10): 0

==================================================
        UNIVERSITY ADMISSION DECISION REPORT
==================================================
  Entrance Score       : 20.0
  GPA                  : 8.0
  Recommendation       : No
  Category             : OBC
  Extracurricular Score: 0.0
--------------------------------------------------
  Bonus Applied        : None
  Effective Score      : 20.0
--------------------------------------------------
  Result  =>  REJECTED
  Reason  : Effective score 20.0 is below the OBC cutoff of 65 (original score 20.
0, bonus +0).
==================================================

Evaluate another student? (yes/no):
```

Part B:- Income Tax Calculator

File Link:-
https://github.com/NiranjanHebli/Day8-AM-Assigment/blob/main/income_tax_calculator.py

Output Screenshots:-

```
(base) niranjanhebli@192 Day8-AM-Assigment % python3 income_tax_calculator.py

+-------------------------------------------------------------+
|   INCOME TAX CALCULATOR — India New Regime FY 2024-25   |
+-------------------------------------------------------------+
  Slabs: 0%(0-3L)   |

         5%(3-7L)   |

         10%(7-10L) |

         15%(10-12L)|

         20%(12-15L)|

         30%(>15L)  |

  Standard Deduction: ₹75,000  |  Rebate 87A: ≤ ₹7L

+-------------------------------------------------------------+
Enter your annual gross income (₹): 250000
```

```
(base) niranjanhebli@192 Day8-AM-Assigment % python3 income_tax_calculator.py
         30%(>15L)  |

  Standard Deduction: ₹75,000  |  Rebate 87A: ≤ ₹7L

+-------------------------------------------------------------+
Enter your annual gross income (₹): 250000

═══════════════════════════════════════════════════════════════
     INDIAN INCOME TAX CALCULATOR — NEW REGIME FY 2024-25
═══════════════════════════════════════════════════════════════
  Gross Annual Income           :            ₹2,50,000
  Less: Standard Deduction      :               ₹75,000
  Taxable Income                :            ₹1,75,000
───────────────────────────────────────────────────────────────
  SLAB                INCOME IN SLAB   RATE            TAX
  ──────────────────  ──────────────   ────   ──────────────
  ₹0 — ₹3L                ₹1,75,000     0%               —
───────────────────────────────────────────────────────────────
  Tax Before Rebate             :                     ₹0
  Rebate u/s 87A (income ≤ 7L)  : -                   ₹0
  Tax After Rebate              :                     ₹0
  Health & Education Cess (4%)  :                     ₹0
═══════════════════════════════════════════════════════════════
  TOTAL TAX PAYABLE             :                     ₹0
  Effective Tax Rate            :                  0.00%
═══════════════════════════════════════════════════════════════

Calculate for another income? (yes/no):
```

```
(base) niranjanhebli@192 Day8-AM-Assigment % python3 income_tax_calculator.py
  Standard Deduction: ₹75,000  |  Rebate 87A: ≤ ₹7L

+------------------------------------------------------------+
Enter your annual gross income (₹): 1500000

================================================================
     INDIAN INCOME TAX CALCULATOR — NEW REGIME FY 2024-25
================================================================
  Gross Annual Income           :           ₹15,00,000
  Less: Standard Deduction      :               ₹75,000
  Taxable Income                :           ₹14,25,000
----------------------------------------------------------------
  SLAB                INCOME IN SLAB   RATE            TAX
  ------------------  --------------  -----  -------------
  ₹0 — ₹3L                 ₹3,00,000     0%              —
  ₹3L — ₹7L                ₹4,00,000     5%         ₹20,000
  ₹7L — ₹10L               ₹3,00,000    10%         ₹30,000
  ₹10L — ₹12L              ₹2,00,000    15%         ₹30,000
  ₹12L — ₹15L              ₹2,25,000    20%         ₹45,000
----------------------------------------------------------------
  Tax Before Rebate             :            ₹1,25,000
  Health & Education Cess (4%)  :                ₹5,000
================================================================
  TOTAL TAX PAYABLE             :            ₹1,30,000
  Effective Tax Rate            :                8.67%
================================================================

Calculate for another income? (yes/no):
```

Part C:- Interview Ready

Q1. Conceptual Design

Multiple “if” statements run every “if” block if its condition is true, regardless of prior results. In contrast, “elif” (else if) only evaluates if previous conditions failed, ensuring mutual exclusivity within the chain.

Example Code:-

Consider this input: score = 85.

Multiple if version:

—----------

```
if score > 80:
    print("Grade: B")
if score > 70:
    print("Grade: C")
if score > 60:
    print("Grade: D")
```

—----------

elif version:

—----------

```
if score > 90:
    print("Grade: A")
elif score > 80:
    print("Grade: B")
elif score > 70:
    print("Grade: C")
```

—----------

Outputs:-

Approach	**Output**	
Multiple If	Grade: B Grade: C Grade: D	
elif	Grade: B	

Why the Difference:-

Multiple if evaluates all conditions separately, so 85 > 80 (true) and 85 > 70 (true) both trigger prints. elif skips remaining checks after the first true condition (85 > 80), preventing further execution for efficiency and logical branching

Q2. Coding

File Link:-
https://github.com/NiranjanHebli/Day8-AM-Assigment/blob/main/classify_triangle.py

Output Screenshot:-

```
(base) niranjanhebli@192 Day8-AM-Assigment % python3 classify_triangle.py
Enter the lengths of the sides of the triangle (separated by spaces): 10 20 30
The triangle is classified as: Not a triangle
(base) niranjanhebli@192 Day8-AM-Assigment % python3 classify_triangle.py
Enter the lengths of the sides of the triangle (separated by spaces): 3 4 5
The triangle is classified as: Scalene
(base) niranjanhebli@192 Day8-AM-Assigment % python3 classify_triangle.py
Enter the lengths of the sides of the triangle (separated by spaces): 3 3 3
The triangle is classified as: Equilateral
(base) niranjanhebli@192 Day8-AM-Assigment % python3 classify_triangle.py
Enter the lengths of the sides of the triangle (separated by spaces): 3 3 4
The triangle is classified as: Isosceles
(base) niranjanhebli@192 Day8-AM-Assigment % python3 classify_triangle.py
Enter the lengths of the sides of the triangle (separated by spaces): 10 20 30
The triangle is classified as: Not a triangle
(base) niranjanhebli@192 Day8-AM-Assigment % python3 classify_triangle.py
Enter the lengths of the sides of the triangle (separated by spaces): -1 20 30
The triangle is classified as: Not a triangle
(base) niranjanhebli@192 Day8-AM-Assigment %
```

Q3. Debug / Analyze

What is the bug:-

The bug is that multiple independent if statements overwrite the grade variable repeatedly, so only the final true condition persists.

For score = 85, the code sets grade = 'D' (85≥60 true), then 'C' (85≥70 true), then 'B' (85≥80 true), but skips 'A' (85≥90 false). Because of this, more than one condition can be true, and later conditions overwrite earlier ones.

Why Wrong Output:-

It does not give the wrong output, the only thiing is the comparison is done in reverse order

Correct Fix:-

Replace with if-elif-else chain in descending order:-

```
score = 85
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'
print(grade)
```

This ensures only the highest matching condition assigns grade, stopping further checks.

Part D:- AI-Augmented Task

Prompt:-

Write a detailed Python program that validates an Indian PAN (Permanent Account Number) card format using only if-else conditions (no regex).

Requirements:

- Input: A string representing the PAN (e.g., "ABCDE1234F")
- Valid PAN format: Exactly 10 characters - first 5 uppercase letters (A-Z), next 4 digits (0-9), last 1 uppercase letter (A-Z)
- Additional validation: 4th character must be one of: 'P', 'C', 'H', 'F', 'A', 'T', 'B', 'L', 'J', 'G' (taxpayer type codes)
- Output: Print "Valid PAN" or "Invalid PAN" with specific error messages for different failure cases

Provide:

1. Complete working code with proper input handling
2. Test cases showing valid PANs (like "ABCDE1234F") and invalid ones (wrong length, lowercase, wrong characters, invalid 4th character)
3. Clear comments explaining the if-else logic flow

AI - Generated Code:-

```python
# PAN Validation Program using only if-else (No Regex)

# Function to validate PAN
def validate_pan(pan):

    # Rule 1: Length must be exactly 10
    if len(pan) != 10:
        print("Invalid PAN: Length must be exactly 10 characters.")
        return

    # Rule 2: First 5 characters must be uppercase letters
    for i in range(5):
        ch = pan[i]
        if not (ch >= 'A' and ch <= 'Z'):
            print(f"Invalid PAN: Character {i+1} must be an uppercase letter (A-Z).")
            return

    # Rule 3: 4th character must be valid taxpayer type
    valid_types = ['P', 'C', 'H', 'F', 'A', 'T', 'B', 'L', 'J', 'G']
    if pan[3] not in valid_types:
        print("Invalid PAN: 4th character must be a valid taxpayer type code.")
        return

    # Rule 4: Next 4 characters must be digits
    for i in range(5, 9):
        ch = pan[i]
        if not (ch >= '0' and ch <= '9'):
            print(f"Invalid PAN: Character {i+1} must be a digit (0-9).")
            return

    # Rule 5: Last character must be uppercase letter
    if not (pan[9] >= 'A' and pan[9] <= 'Z'):
        print("Invalid PAN: Last character must be an uppercase letter (A-Z).")
        return

    # If all conditions pass
    print("Valid PAN")


# ===== Main Program =====

pan_input = input("Enter PAN number: ")
validate_pan(pan_input)
```

Are All Positions Validated Correctly?

Yes, every position is properly validated. The program checks the length, ensures the first five characters are uppercase letters, verifies the fourth character against valid taxpayer codes, confirms the next four are digits, and checks the last character is an uppercase letter. The layered validation makes the logic complete and accurate.

Are Edge Cases Handled?

Most edge cases are handled, including wrong length, lowercase letters, misplaced digits, special characters, and invalid fourth-character codes. A minor improvement would be trimming whitespace using strip() before validation.

Is the Approach Correct (Character-by-Character vs Regex)?

Yes, the character-by-character approach is correct and suitable, especially since regex is not allowed. It improves clarity, debugging, and allows specific error messages, making it ideal for interviews or learning purposes.

Is the Code Pythonic?

The code is readable and logically structured. It could be slightly more Pythonic by using chained comparisons, sets for membership checks, or built-in methods like isupper() and isdigit(). However, it is still clean and appropriate for the given constraints

Improvised Version:-.

```
# Improved PAN Validation Program (No Regex)

def validate_pan(pan):

    # Remove leading/trailing spaces
    pan = pan.strip()

    # Rule 1: Length check
    if len(pan) != 10:
        print("Invalid PAN: Length must be exactly 10 characters.")
        return

    # Rule 2: First 5 characters must be uppercase letters
    for i in range(5):
        if not ('A' <= pan[i] <= 'Z'):
            print(f"Invalid PAN: Character {i+1} must be an uppercase letter (A-Z).")
            return

    # Rule 3: 4th character must be valid taxpayer type
    valid_types = {'P', 'C', 'H', 'F', 'A', 'T', 'B', 'L', 'J', 'G'}
    if pan[3] not in valid_types:
        print("Invalid PAN: 4th character must be a valid taxpayer type code.")
        return

    # Rule 4: Next 4 characters must be digits
    for i in range(5, 9):
        if not ('0' <= pan[i] <= '9'):
            print(f"Invalid PAN: Character {i+1} must be a digit (0-9).")
            return

    # Rule 5: Last character must be uppercase letter
    if not ('A' <= pan[9] <= 'Z'):
        print("Invalid PAN: Last character must be an uppercase letter (A-Z).")
        return
```

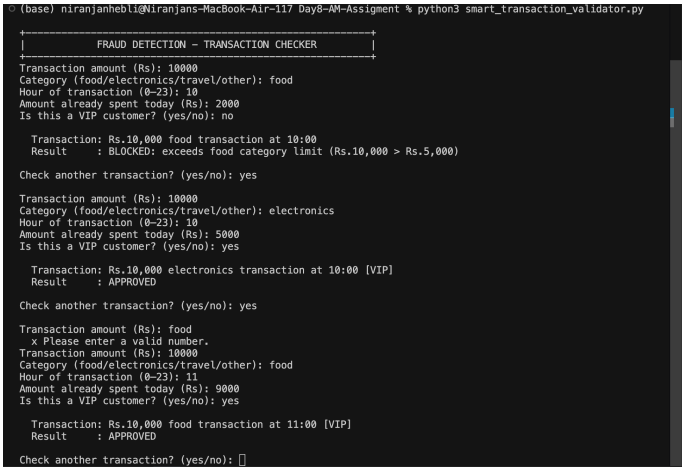
```
    # If all checks pass
    print("Valid PAN")


# ===== Main Program =====
pan_input = input("Enter PAN number: ")
validate_pan(pan_input)
```

Part E:- Smart Transaction Validator

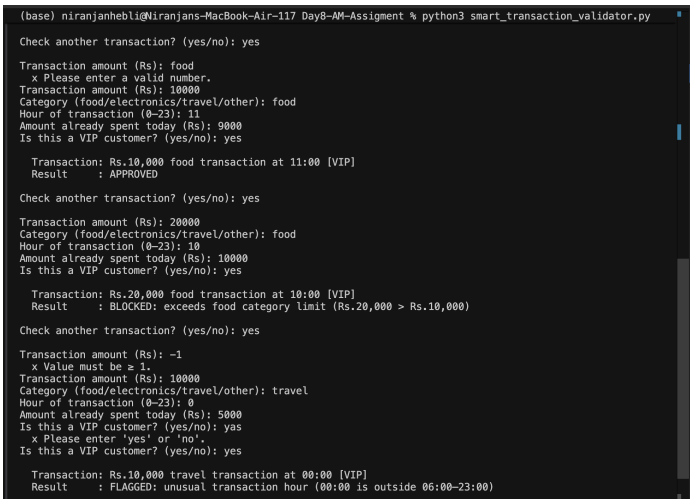
File Link:-

https://github.com/NiranjanHebli/Day8-AM-Assigment/blob/main/smart_transaction_validator.py

Output Screenshots:-



```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-AM-Assigment % python3 smart_transaction_validator.py

+------------------------------------------------------------+
|            FRAUD DETECTION - TRANSACTION CHECKER           |
+------------------------------------------------------------+
Transaction amount (Rs): 10000
Category (food/electronics/travel/other): food
Hour of transaction (0-23): 10
Amount already spent today (Rs): 2000
Is this a VIP customer? (yes/no): no

  Transaction: Rs.10,000 food transaction at 10:00
  Result     : BLOCKED: exceeds food category limit (Rs.10,000 > Rs.5,000)

Check another transaction? (yes/no): yes

Transaction amount (Rs): 10000
Category (food/electronics/travel/other): electronics
Hour of transaction (0-23): 10
Amount already spent today (Rs): 5000
Is this a VIP customer? (yes/no): yes

  Transaction: Rs.10,000 electronics transaction at 10:00 [VIP]
  Result     : APPROVED

Check another transaction? (yes/no): yes

Transaction amount (Rs): food
  x Please enter a valid number.
Transaction amount (Rs): 10000
Category (food/electronics/travel/other): food
Hour of transaction (0-23): 11
Amount already spent today (Rs): 9000
Is this a VIP customer? (yes/no): yes

  Transaction: Rs.10,000 food transaction at 11:00 [VIP]
  Result     : APPROVED

Check another transaction? (yes/no):
```



```
(base) niranjanhebli@Niranjans-MacBook-Air-117 Day8-AM-Assigment % python3 smart_transaction_validator.py

Check another transaction? (yes/no): yes

Transaction amount (Rs): food
  x Please enter a valid number.
Transaction amount (Rs): 10000
Category (food/electronics/travel/other): food
Hour of transaction (0-23): 11
Amount already spent today (Rs): 9000
Is this a VIP customer? (yes/no): yes

  Transaction: Rs.10,000 food transaction at 11:00 [VIP]
  Result     : APPROVED

Check another transaction? (yes/no): yes

Transaction amount (Rs): 20000
Category (food/electronics/travel/other): food
Hour of transaction (0-23): 10
Amount already spent today (Rs): 10000
Is this a VIP customer? (yes/no): yes

  Transaction: Rs.20,000 food transaction at 10:00 [VIP]
  Result     : BLOCKED: exceeds food category limit (Rs.20,000 > Rs.10,000)

Check another transaction? (yes/no): yes

Transaction amount (Rs): -1
  x Value must be ≥ 1.
Transaction amount (Rs): 10000
Category (food/electronics/travel/other): travel
Hour of transaction (0-23): 0
Amount already spent today (Rs): 5000
Is this a VIP customer? (yes/no): yas
  x Please enter 'yes' or 'no'.
Is this a VIP customer? (yes/no): yes

  Transaction: Rs.10,000 travel transaction at 00:00 [VIP]
  Result     : FLAGGED: unusual transaction hour (00:00 is outside 06:00-23:00)
```